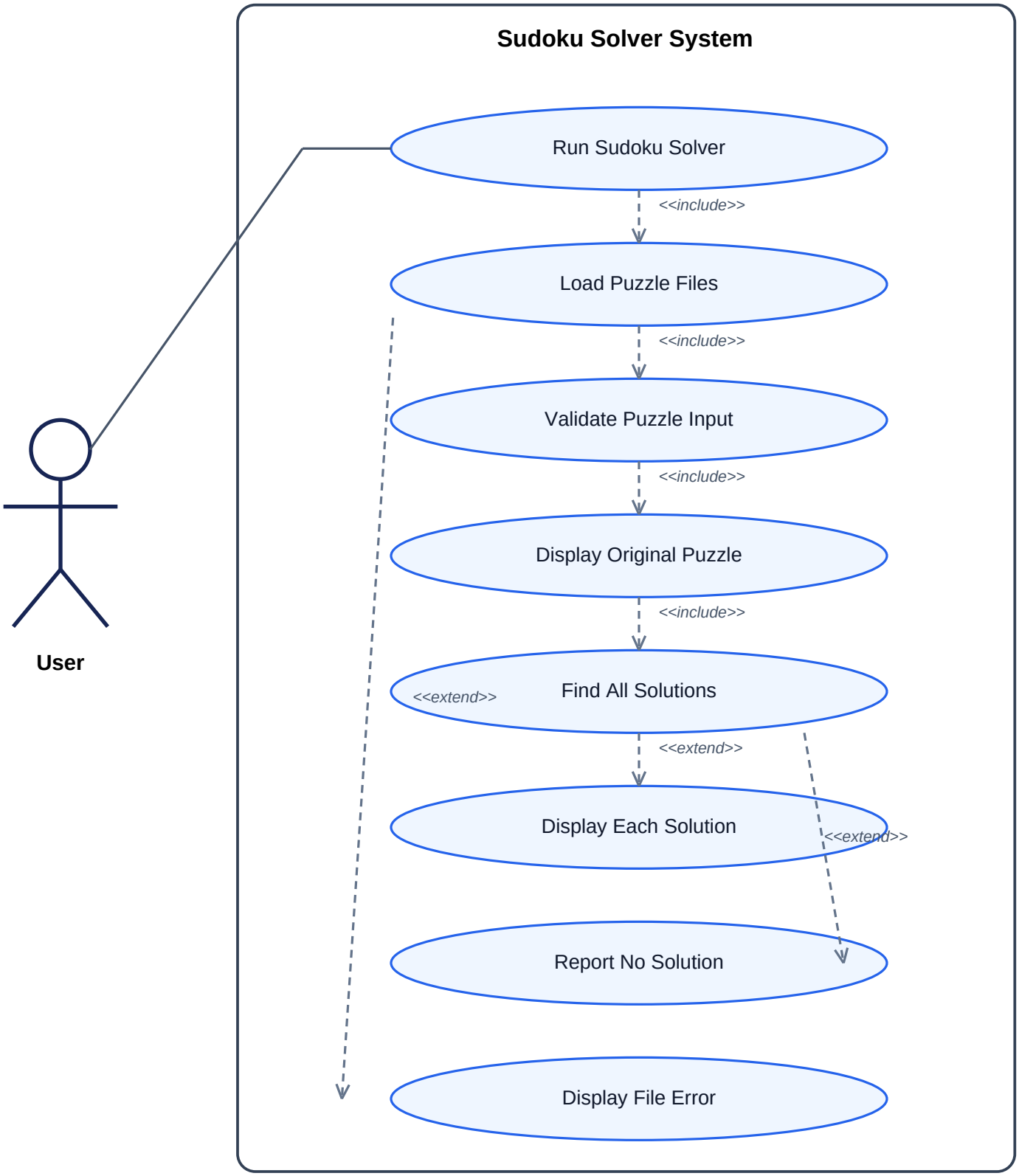
# Sudoku Solver - UML Use Case Diagram

EECS 348 Assignment 4 | System behavior




Dashed arrows show included or conditional behavior.

The program automatically processes puzzle1.txt through puzzle5.txt.

# Sudoku Solver - Use Case Documentation

## 1. Identify / Discover a Use Case

Primary use case: **Solve Sudoku Puzzles**. Primary actor: **User**. System: **Sudoku Solver**. Supporting use cases include loading and validating puzzle files, displaying the original puzzle, searching for solutions, displaying every solution, and reporting errors or unsolvable puzzles.

## 2. Brief Description

The user runs a C++ Sudoku solver. The system reads five text files containing 9x9 Sudoku grids, prints each original puzzle, and searches for all valid solutions using recursive backtracking. It displays each solution or an appropriate message when the puzzle is invalid or cannot be solved.

## 3. Step-by-Step Flow of Events

1. The user starts the compiled Sudoku Solver program.

2. The system creates a Sudoku object for each file, puzzle1.txt through puzzle5.txt.

3. The system attempts to open and read the file, expecting exactly 81 cells containing digits 1-9 or underscores.

4. If loading fails, the system prints a specific file error and continues with the next puzzle.

5. For a successfully loaded file, the system displays the original Sudoku grid.

6. The system checks that existing numbers do not conflict within rows, columns, or 3x3 blocks.

7. If the starting grid is invalid, the system reports that no solution was found and continues.

8. Otherwise, the system recursively chooses an empty cell with the fewest valid candidates, tries digits, and backtracks as needed.

9. Each time a complete valid grid is reached, the system displays that solution and increments the solution count.

10. When the search ends, the system prints the total number of solutions; if zero, it prints "No solution found".

11. The system repeats for the remaining files and then exits.

## 4. Detailed Use Case Description

**Use case name:** Solve Sudoku Puzzles
**Primary actor:** User
**Trigger:** User executes Assignment4.exe.
**Preconditions:** The program is compiled; puzzle files are expected in its working directory.
**Main success scenario:** A puzzle file is read and validated, the original grid is shown, recursive backtracking finds valid completed grids, and all solutions and their count are displayed.
**Alternate flows:** Missing file, incomplete grid, invalid character, or extra data produces a specific error

message. Conflicting starting clues or an unsolvable grid produce a no-solution result. A puzzle with multiple solutions prints each one.
**Postconditions:** Every available puzzle has been processed; no input puzzle file is modified.
**Business rules:** Each completed row, column, and 3x3 block contains digits 1 through 9 exactly once. Underscores represent blank cells.